

Web Development Cheat Sheet

HTML BASICS	CSS BASICS	CSS SELECTORS & PROPERTIES
Document Structure <!DOCTYPE html> <html> <head> <meta charset='UTF-8'> <title>...</title> </head> <body>...</body> </html> Common Tags <h1>-<h6> Headings <p> Paragraph Link Image <div> Division Inline container / Lists List item <table> Table <form> Form <input> Input field <button> Button Attributes id='unique' Unique ID class='name' Class style='...' Inline CSS data-*='...' Custom data	Linking CSS <link rel='stylesheet' href='style.css'> Inline CSS <style> selector { prop: val; } </style> Box Model margin Outside space border Element border padding Inside space width/height Content Display display: block; display: inline; display: inline-block; display: flex; display: grid; display: none; Position position: static; position: relative; position: absolute; position: fixed; position: sticky;	Basic Selectors element Tag name .class Class #id ID * Universal Combinators A B Descendant A > B Child A + B Adjacent sibling A ~ B General sibling Common Properties color Text color background-color font-size, font-family text-align margin, padding border width, height Flexbox justify-content align-items flex-direction flex-wrap
JAVASCRIPT BASICS	JS DOM & EVENTS	JAVASCRIPT FUNCTIONS
Variables let x = 5; Block scope const y = 10; Constant var z = 3; Function scope Data Types String 'text' or "text" Number 42, 3.14 Boolean true/false Array [1, 2, 3] Object {key: value} null, undefined Operators +, -, *, /, % Math ==, ===, !=, !== <, >, <=, >= &&, , ! Logical Conditionals if (condition) {} else if (condition) {} else {} condition ? a : b	Selecting Elements getElementById('id') querySelector('.class') querySelectorAll('div') getElementsByClassName() Manipulating DOM element.textContent element.innerHTML element.style.prop element.classList.add() element.setAttribute() Events addEventListener(type, callback) Common: click, submit, change, keypress, load Creating Elements createElement('div') appendChild(element) removeChild(element)	Function Declaration function name(params) { return value; } Function Expression const fn = function() { // code }; Arrow Functions const fn = () => {} const fn = x => x * 2 const fn = (a,b) => a+b Loops for (let i=0; i<10; i++) while (condition) {} arr.forEach(item => {}) for (let item of arr) Array Methods map(), filter() reduce(), find() push(), pop() slice(), splice()

JQUERY	API REQUESTS	ASYNC / PROMISES
Including jQuery <script src='jquery.js'> </script> Syntax \$(selector).action() \$(document).ready(fn) \$(() => { /* code */ }) Selectors \$('element') \$('.class') \$('#id') \$('[attribute]') Events \$(sel).click(fn) \$(sel).on('event', fn) \$(sel).hover(fnIn, fnOut) Manipulation .text(), .html() .val() .attr(), .css() .addClass() .append(), .prepend() .remove(), .empty() Effects .show(), .hide() .fadeIn(), .fadeOut() .slideDown(), .slideUp()	XMLHttpRequest (Old) const xhr = new XMLHttpRequest(); xhr.open('GET', url); xhr.send(); Fetch API (Modern) fetch(url) .then(res => res.json()) .then(data => {...}) .catch(err => {...}); GET Request fetch(url) .then(r => r.json()) .then(d => console.log(d)) POST Request fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) }) jQuery AJAX \$.ajax({ url: 'api/data', method: 'GET', success: fn, error: fn });	Promises new Promise((resolve, reject) => { if (success) resolve(data); else reject(error); }) Promise States • Pending: Initial state • Fulfilled: Success • Rejected: Failure .then() Chaining promise .then(result => { return process(result); }) .then(data => {...}) .catch(err => {...}) .finally(() => {...}); Async/Await async function getData() { try { const res = await fetch(url); const data = await res.json(); return data; } catch (err) { console.error(err); } }
PROMISE METHODS	ASYNC PATTERNS	COMMON GOTCHAS
Promise.all() Promise.all([p1, p2, p3]) .then(results => { // All resolved }); Waits for all promises Fails if any rejects Promise.race() Promise.race([p1, p2]) .then(result => { // First to settle }); Returns first settled Promise.allSettled() Promise.allSettled([...]) .then(results => { // All finished }); Waits for all, never fails Promise.any() Promise.any([p1, p2]) .then(result => { // First fulfilled }); First successful resolve	Sequential Execution async function seq() { const a = await task1(); const b = await task2(); return [a, b]; } Parallel Execution async function parallel() { const [a, b] = await Promise.all([task1(), task2()]); return [a, b]; } Error Handling // With try/catch try { await asyncFn(); } catch (e) {} // With .catch() asyncFn() .catch(e => {...});	Await in Loops // Sequential (slow) for (let i of items) { await process(i); } // Parallel (fast) await Promise.all(items.map(i => process(i))); Forgotten await // Wrong const data = fetch(url); Returns Promise, not data // Correct const data = await fetch(url); Async Function Returns async function fn() { return 'value'; } Always returns Promise fn().then(v => {...});